

wizard's coins. In addition, if we later wanted to add a `weightInGrams()` method for the wizard's weight, that method name would already be taken.

It's much better to have a `WizCoin` object as an attribute, because a wizard customer "has a" quantity of wizard coins:

```
import wizcoin

class WizardCustomer:
    def __init__(self, name):
        self.name = name
    ❶ self.purse = wizcoin.WizCoin(0, 0, 0)

wizard = WizardCustomer('Alice')
print(f'{wizard.name} has {wizard.purse.value()} knuts worth of money.')
print(f'{wizard.name}'s coins weigh {wizard.purse.weightInGrams()} grams.')
```

Instead of making the `WizardCustomer` class inherit methods from `WizCoin`, we give the `WizardCustomer` class a `purse` attribute ❶, which contains a `WizCoin` object. When you use composition, any changes to the `WizCoin` class's methods won't change the `WizardCustomer` class's methods. This technique offers more flexibility in future design changes for both classes and leads to more maintainable code.

Inheritance's Downside

The primary downside of inheritance is that any future changes you make to parent classes are necessarily inherited by all its child classes. In most cases, this tight coupling is exactly what you want. But in some instances, your code requirements won't easily fit your inheritance model.

For example, let's say we have `Car`, `Motorcycle`, and `LunarRover` classes in a vehicle simulation program. They all need similar methods, such as `startIgnition()` and `changeTire()`. Instead of copying and pasting this code into each class, we can create a parent `Vehicle` class and have `Car`, `Motorcycle`, and `LunarRover` inherit it. Now if we need to fix a bug in, say, the `changeTire()` method, there's only one place we need to make the change. This is especially helpful if we have dozens of different vehicle-related classes inheriting from `Vehicle`. The code for these classes would look like this: